Decision Analytics Journal 9 (2023) 100360

Contents lists available at ScienceDirect

Decision Analytics Journal

journal homepage: www.elsevier.com/locate/dajour

A step-size follow-the-leader optimization algorithm with an improved step parameters

Priyanka Singh [a,*], Rahul Kottath [b,c]

[a] School of Engineering and Science, SRM University-AP, Amravati, 522502, India
[b] School of Electrical and Electronics Engineering, VIT Bhopal University, Bhopal, India
[c] Digital Tower, Bentley Systems India Private Limited, Pune, India

ARTICLE INFO

Keywords:
Follow-the-leader algorithm
Metaheuristic optimization
Step-size parameter
Benchmark function
Mass prediction
Cost prediction

ABSTRACT

Follow the leader (FTL) algorithm is a newly developed optimization algorithm inspired by a sheep's movement within a flock. FTL has been successfully implemented to solve power prediction problems. However, the probability of falling in local optima is high due to randomness in the step parameter. This paper proposes a step-size follow-the-leader (SFTL) algorithm with decreasing and increasing step-size combinations. The improved step parameter tunes the search space by generating a new solution to improve the accuracy and convergence rate of the FTL algorithm. Four different FTL variants have been presented in this paper to show the impact of the dynamic step-size parameter. FTL improvement is verified by testing SFTL over thirty-two fixed unimodal, unimodal, fixed multimodal, and multimodal benchmark functions. The computational results indicate that SFTL significantly improves the basic FTL algorithm and converges early compared to other algorithms. SFTL has also been tested on five real engineering design problems and obtained results show that SFTL outperformed FTL and other popular optimization algorithms.

1. Introduction

For the past few years, optimization has become an active research topic in engineering. However, most of these engineering problems are complex and multi-modal, which cannot be solved by traditional optimization algorithms [1]. One of the solutions to such problems is metaheuristic algorithms. Most of the metaheuristic approaches are swarm-based, which starts with random individuals, and these individuals are updated in every iteration to find optimal parameters of a given problem [2]. Metaheuristic optimization approaches have proven itself superior to the traditional optimization approaches. At present, they have been applied to detect trends [3], Electricity price and load forecasting [4–6], drilling process [7], data classification [8], truss optimization [9], surface electromyography [10], complex engineering problems [11–14], and many more. Metaheuristic algorithms can be broadly classified into single solution-based algorithms and population-based algorithms [15]. Solution-based algorithms start with a single random solution and are updated during their optimization phase, such as Simulated Annealing [16] and Tabu Search [17]. However, population-based algorithms start with a random population to evolve in the upcoming generation of its optimization process [18]. Generally speaking, most population-based algorithms are inspired by the natural phenomenon where the current population updates itself using natural operators to generate the next population. Further, they can be grouped into evolution-based, swarm-based, physics-based, and human-based algorithms [19]. A comprehensive collection of different optimization algorithms has been shown in Fig. 1.

1.1. Evolution-based algorithms

Evolution-based algorithms are bio-inspired algorithms that imitate the natural phenomena of evolution [20]. They are expected to generate the right solutions, i.e., near to optimal solution for problems that cannot be resolved by traditional methods [21]. In EAs, "survival-of-fittest theory is used to generate next population". Genetic Algorithm (GA) is one of the first evolutionary algorithms that mimic the concept of natural evolution using crossover and mutation operators [22]. Differential Evolution (DE) generates new individuals by selecting two random individuals from the current population apart from crossover and mutation operators [23]. Metawa et al. developed a dynamic bank lending decision model by utilizing GA to maximize the profit [24]. Terán et al. developed a novel approach to optimize coordination mechanisms between agents for modeling industrial automation systems using cultural algorithms [25]. Maza and Touahria presented a multi-objective feature selection algorithm using estimation of distribution algorithm for intrusion detection system [26]. Recently, Deng et al. proposed an optimal mutation strategy to improve the DE algorithm

* Corresponding author.
E-mail addresses: priyanka.si@srmap.edu.in (P. Singh), rahulkottath@gmail.com (R. Kottath).

https://doi.org/10.1016/j.dajour.2023.100360
Received 2 August 2023; Received in revised form 27 October 2023; Accepted 5 November 2023
Available online 7 November 2023
2772-6622/© 2023 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

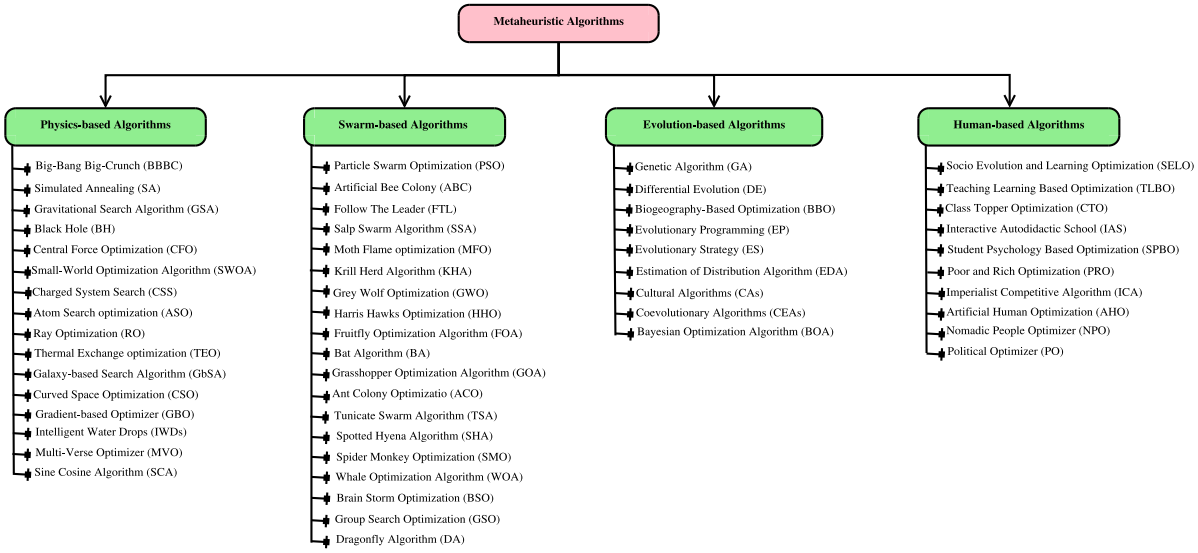

Fig. 1. Classification of metaheuristic algorithms.

with the wavelet basis function to solve the airport gate assignment problem [27].

1.2. Swarm-based algorithms

Swarm-based algorithms imitate the collective social behavior of creatures living in flocks, swarms, herds, and schools. This behavioral intelligence is developed when swarms interact with each other and their environment [20]. Particle Swarm Optimization (PSO), proposed by Eberhart and Kennedy, is the most widely used SA, which is inspired by the foraging behavior of bird flocks or fish school [28]. The Artificial Bee Colony (ABC) algorithm mimics the foraging behavior and information sharing ability of honeybee swarms [29]. Follow the Leader algorithm is based on sheep movement in a flock during foraging [30]. The Salp Swarm Optimization (SSA) algorithm is inspired by the navigating and foraging behavior of salps in the ocean [22]. Moth Flame Optimization (MFO) mimics moth movement using the transverse orientation technique for navigation by maintaining a fixed angle with artificial light to find their path [31]. El et al. proposed a hybrid optimization algorithm by combining SSA and PSO optimization algorithms to optimize power flow [32]. Yildiz et al. presented MFO to get maximum profit rate for the milling optimization problem of manufacturing industries [31]. Ateya et al. presented a new chaotic SSA algorithm to locate a minimal number of controllers and connections needed between switches and controllers in software-defined networks [33]. Yildiz et al. used three different optimization algorithms for grinding operations to find the optimal processing parameters [34].

1.3. Physics-based algorithms

The development of physics-based algorithms is motivated by the laws of physics [15]. Individuals share their information and navigate as per the rules of physics, e.g., gravitational force, laws of cooling, laws of motion, electromagnetic force, inertia force, equilibrium, and movement of an electron [35]. The basis of Thermal Exchange Optimization (TEO) algorithm depends on Newton's law of cooling [36]. The Black Hole (BH) algorithm imitates the physics of black hole behavior of pulling-in surrounding stars [37]. Simulated Annealing (SA) is based on the annealing process of an iron cast when cooled from high to low-temperature [38]. Kaveh and Dadras proposed a novel approach to the TEO algorithm for the identification of vibration-based structural damage [36]. Abdulwahab et al. proposed a model based on the Levy distribution considered BH algorithm. [37]. Levy distribution helps individuals to make long jumps and ensure the exploration of search space against the BH algorithm.

1.4. Human-based algorithms

Human-based algorithms imitate human behavior and their characteristics [15]. Class Topper Organization algorithm is based on students' competitive behavior in the same class of a school. This intelligence behavior among students makes them improve their performance in class and can be modeled to solve complex optimization problems [39]. Poor and Rich Optimization depicts the behavior of two groups of people staying in society. The algorithm portrays the effort given by rich and poor people to earn wealth and always improve their financial status. The rich group always tries to maximize their class gap relating to the poor group, while the poor group tries to achieve wealth by modeling rich people [40]. Student Psychology Based Optimization algorithm is based on the psychological behavior of the students who are trying to get marks closer to the best student of the class in the examination [41].

Metaheuristic algorithms use a different approach to generate an optimal solution, amongst which diversification and intensification are considered important aspects of search strategies [42]. In the former, the capability of the algorithm is measured by how well it can explore the global search space to avoid getting local solutions, while in the latter, the algorithm should be able to perform locally by exploring nearby favorable solutions. Therefore, a proper balance between the two is needed, which is a huge challenge for any metaheuristic algorithms [9,19]. Some of the literature experiments improve the performance of basic algorithms by choosing appropriate control parameters or developing hybrid models. Moreover, according to the "No-Free-Lunch" theorem, no metaheuristic algorithm can solve all optimization problems [23]. One algorithm can perform well for some problems, while it may generate poor results for some others. Consequently, the development of an efficient metaheuristic algorithm with a balanced trade-off between diversification and intensification is still an open problem [43].

As discussed earlier, choosing an appropriate control parameter may improve the trade-off between diversification and intensification. The larger the fixed step size, the more is diversification, and the lesser is the intensification and vice versa. The chance of getting into local optima is high for a smaller step size value. Hu et al. presented a decreasing step size method named sigmoid function instead of fixed step size in Fruit Fly optimization algorithm (FFOA) to improve the efficacy of basic FFOA [44]. Mareli and Twala proposed three cuckoo search algorithms based on dynamically increasing step-size parameters (linearly increasing, exponentially increasing, and increasing with the

power of three) for an increasing number of generations [45]. Yu et al. presented a new dynamic controlled scheme for step to balance global diversification and local intensification [46].

FTL algorithm is a recent metaheuristic optimization algorithm that simulates the foraging behavior of sheep. Among several species of sheep, Merino sheep show a high correlation between the position in a moving flock. Merino sheep always move as a single flock and exhibit social dominance. Squires and Daws [47] assessed Merinos and Border Leicesters sheep to study leadership behavior when animals move in their flock or herd. The result showed that Merino sheep have a fewer number of leader sheep than Border Leicesters. Leaders in the Merino flock generate a quicker response than other sheep, thereby influencing flock behavior. This leadership and social dominance relationship shown by the sheep has been mathematically modeled and implemented to solve short-term electricity load forecasting [30]. Later, a multi-objective FTL was proposed to simultaneously improve the forecasting accuracy and stability of the model [4]. The foundation of the FTL algorithm lies in the leadership and social dominance behavior of sheep. This flocking behavior drives every sheep to follow the steps of their leader. The simplicity in implementation and high convergence rate makes the FTL algorithm suitable for different optimization problems. One of the limitations of FTL is that it is a parametric algorithm where a suitable value of inertia factor ! has to be chosen. For the general case of FTL, the ! value is chosen randomly between 0 and 1 for every generation. As the movement or step taken by an individual directly depends on this !, it is quite important to find a solution that can avoid the stagnation problem. As discussed earlier, an appropriate solution for balanced diversification and intensification would be to choose parameters using a dynamic approach.

In this paper, we modify the basic FTL algorithm with different ! values to provide a dynamic step named step-size follow-the-leader (SFTL). Four variants of FTL have been implemented to find a suitable ! value. The results of variants are compared with basic FTL to analyze the impact of step-size over benchmark functions. Subsequently, the competence of SFTL is shown over complex engineering design problems. The results dominate significantly from compared algorithms. The major contributions of our work are as follows:

- An improved FTL optimization algorithm with step-size named SFTL has been proposed.
- Four dynamic step parameters have been presented to improve the FTL algorithm.
- The proposed approach SFTL is tested on thirty-two unimodal and multimodal benchmark functions.
- Further, SFTL is implemented to tackle various engineering test problems.

The paper is organized in the following manner. Section 2 provides the related literature on optimization algorithms. Section 3 presents the improved FTL algorithm, and Section 4 describes the validation of SFTL over thirty-two benchmark functions. Section 5 describes the implementation of SFTL on engineering problems. Finally, Section 6 analyzes the experimental results, and Section 7 infers the work with some future work.

2. Related works

In this section, we discuss various literature related to this work. The section describes different optimization algorithm and their applications in the real world.

Various optimization algorithms have evolved over the period for solving engineering problems. Some of the traditional engineering applications include automatic control systems, electronics circuit design, product and shape design, data analysis, robotics, and statistical modeling. Most of these applications require optimization of design parameters, maximization of profit, and minimization of cost or materials. Moreover, optimization algorithm has been successfully implemented in many forecasting applications such as wind energy forecasting [48], wind speed forecasting [49], electricity load forecasting [30], electricity price forecasting [50], streamflow forecasting for water management applications [51] etc. Recently, salp swarm algorithm has been combined with a flower pollination algorithm for forecasting COVID-19 cases in China [52]. For time-series forecasting applications, various traditional models have been used whose parameters are tuned using optimization algorithms. SAs are highly accepted population-based approach as it provides an option to select the best from multiple possible solutions.

Optimization has been widely used in robotics applications to improve navigation aspects such as perception, localization, path planning, and motion control. Environmental perception is important for the robot to understand the obstacles and act according to the variations. Kubota et al. proposed a perception model that uses GA for obtaining dynamic environment information [53]. Localization applications of optimizations include simultaneous localization and mapping [54], vehicle tracking applications [55,56] etc. Qu et al. proposed a hybrid model by integrating a Grey Wolf Optimizer (GWO) with a modified Symbiotic Organism Search algorithm for planning the path for an unmanned aerial vehicle mission planning system [57]. Contreras-Cruz et al. presented a hybrid model using an Artificial Bee Colony and Evolutionary Programming for path planning [58].

Classical engineering problems are another set of optimization problems that include welding, I-beam, tension/compression spring, pressure vessel, speed reducer, rolling element bearing, three-bar truss, multi-plate disc clutch brake, gear train, and cantilever beam design [59–61]. Marine propeller design is a multi-objective problem where the shape of the marine is optimized by maximizing the efficiency and minimizing the cavitation simultaneously [62]. 2-D airfoil design is another multi-objective problem where the lift force and drag force applied to an airplane are maximized and minimized, respectively [22]. Ong et al. proposed an improved version of the flower pollination algorithm by combining it with chaos theory, inertia weights, and the frog leaping algorithm. The proposed algorithm performed well on benchmark functions and engineering design problems [13]. Similarly, Ghasemi et al. improved global exploration of the Firefly algorithm and tested on benchmark functions and engineering problems [63]. Alam et al. used different metaheuristic algorithms to compare the onset/offset detection performance in surface electromyography, which significantly reduced the manual parameter tuning [10].

The primary reason to use metaheuristics is its ability to approximate highly complex nonlinear functions. This enables the possibility of applying these algorithms to various analytics and prediction applications. Mahmoodi et al. proposed a combination of support vector machines with PSO for stock price forecasting applications that proved itself better than neural networks and cuckoo search [64]. Ong et al. proposed a time series prediction model that improved the performance of wavelet neural networks by generating suitable initial parameters using the Cuckoo Search algorithm [65]. Perla et al. implemented a weighted Water Cycle optimization algorithm for prediction and trend analysis of forex rates using AutoEncoder architecture to enhance its generalization and learning ability [3]. Similarly, metaheuristic algorithms and the Dendritic neuron model have been used to predict crude oil prices to improve the model's performance [66].

From the above literature, we find that metaheuristic algorithms are not only limited to general engineering optimization problems. The application area of metaheuristic algorithms is vast, including analytics and forecasting domains. This work considers five complex design problems for testing the proposed algorithm. Different algorithms are implemented and tested in these problems and compared to evaluate their performance. The improved FTL algorithm with step-size parameters has been discussed in detail in the next section.

Fig. 2. Flowchart of SFTL.

3. FTL algorithm

FTL algorithm is an optimization algorithm inspired by the movement of a sheep within a flock [30]. Every sheep explore green grass in the field (search area), which leads to diversification. Individual sheep try to get closer to the leader surrounded by more green grass and get away from the rear surrounded by less or dry grass, which leads to intensification. Mathematically, this can be modeled as given Eq.

$$S_{next} = S_c + ! * r_1 * (S_{leader} - S_{rear}) \quad (1)$$

where, S_{next} and S_c are the next and current position of sheep, S_{leader} and S_{rear} are current position of leader and rear sheep. $r_1 \in [0; 1]$ and $! \in [0; 1]$, $!$ represents step parameter known as inertia weight. As discussed earlier, every metaheuristic algorithm must balance exploration and exploitation. In FTL, the leader is responsible for intensification while r_1 and $!$ variable help in diversification. However, there is still scope for improvement in optimization algorithms as no such algorithm can outperform all optimization problems [23]. The selection of appropriate control parameters is critical for the better performance of the algorithm. Based on this concept, we have proposed four variants of the FTL algorithm using a different dynamic approach to obtain step-size, $!$.

3.1. Step-size improvement

Most optimization algorithms presented in the literature are parametric. These parameters need to be appropriately tuned before implementing different real-life problems. A step-size parameter has been used in most optimization algorithms to provide inertia to the solution. Hu et al. [44] experimentally verified that the dynamic step-size gives a better performance as compared to a fixed value. Dynamic step-size helps the algorithm in global searching and local optimization. This improves the accuracy of the global search and avoids falling into local optima. In basic FTL, the step-size $!$ is chosen randomly between 0 and 1 in every generation. Although it is dynamic, the randomization creates fluctuations in the solution space. Three different dynamic approaches, namely linearly increasing, exponentially increasing, and exponentially decreasing step-size, have been evaluated in this paper. A novel combination of exponentially increasing and decreasing step-size is proposed in this work to extract the complementary aspects of these parameters. The mathematical expression of step-size is as follows:

$$!_1 = L_0 - \frac{L_0}{1 + e \quad 6 - \frac{12 * ite}{Max_{ite}}} \quad (2)$$

$$!_2 = !_{max} * \frac{ite}{Max_{ite}} \quad (3)$$

$$!_3 = !_{max} * e^{\frac{ite}{Max_{ite}}} \quad (4)$$

g where, $!_{max}$ is maximum value of $!$, ite represents current generation, Max_{ite} is maximum generation and L_0 is initial step-size whose value is set to 1 in this paper. Decreasing the step-size value provides a large step in the initial generations that improves the probability of finding an optimal global solution. With increasing generations, the step-size decreases gradually to improve the local optimization [46]. With this approach, the balance between search strategies can be maintained. Contrary to this, increasing step-size provides a small step in initial generations for local exploitation and a large step with increasing generations to provide a chance to the individual for global exploration [45]. Taking advantage of both, we have proposed an adaptive step-size parameter that combines the complementary nature of both $!_1$ and $!_3$.

The flowchart of SFTL algorithm is shown in Fig. 2. SFTL, being a PAs, starts with a random population of size ps. The best and the worst individuals are chosen to update the solution in the next generation. Individuals update their position using Eq. (1), where two different step parameters $!_1$ and $!_3$ are utilized together to generate individuals for the next generations. Two sets of population S_{next1} and S_{next2} are generated using these step parameters. The next population is selected using fitness values of current and generated individuals. This optimization process stops when Max_{ite} is reached, or an optimal solution is found. The algorithm for SFTL is given in Algorithm 1. The